JNI to AKI transformation

V0.1
- Project and Document by - Ruturaj R Raval

Catalog

1. User code Java 1
    1.1. JNI 1
        1.1.1 MainActivity.java 1
    1.2 AKI 1
        1.2.1 index.ts 1
        1.2.2 index.d.ts 2
2. User code CPP 3
    2.1 JNI 3
        2.1.1 native-lib.cpp 3
        2.1.2 test_djinni_wrapper.cpp 3
    2.2 AKI 4
        2.2.1 hello.cpp 4
        2.2.2 test_djinni_wrapper.cpp 5
3. Djinni generated Java 6
    3.1 JNI 6
        3.1.1 ISayHello.java 6
        3.1.2 MyRecord.java 8
        3.1.3 Gender.java 9
    3.2 AKI 9
        3.2.1 index.d.ts 9
        3.2.2 index.ts 11
4. Djinni generated CPP 12
    4.1 JNI 12
        4.1.1 ISayHello.cpp 12
        4.1.2 JNIISayHello.cpp 13
        4.1.3 JNIMyRecord.cpp 15
    4.2 AKI 16
5. Djinni generated HPP for CPP 17
    5.1 JNI 17
        5.1.1 JNIISayHello.h 17
        5.1.2 JNIMyRecord.h 19
        5.1.3 JNIGender.h 20
    5.2 AKI 20
6. Djinni generated HPP for user code 21
    6.1 JNI 21
        6.1.1 ISayHello.h 21
        6.1.2 MyRecord.h 22
        6.1.3 Gender.h 22
    6.2 AKI 23
        6.2.1 ISayHello.h 23
        6.2.2 MyRecord.h 24
        6.2.3 Gender.h 25
7. CMakeLists.txt file 27
    7.1 JNI 27
    7.2 AKI 28
8. Notes 29

1. User code Java

1.1. JNI

1.1.1 MainActivity.java

```
1 usage
public static native void androidJNISayHello(int i);

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    binding = ActivityMainBinding.inflate(getLayoutInflater());
    setContentView(binding.getRoot());

    // Example of a call to a native method
    TextView tv = binding.sampleText;
    tv.setText(stringFromJNI());

    androidJNISayHello( i: 1);

    ISayHello djnniInterface = ISayHello.newInstance();
    MyRecord infoStore = new MyRecord( id: 1234,  name: "djinni test \n",  project: "djinni", Gender.Man);
    String a = djnniInterface.sayHello(infoStore);
    tv.setText(a);
}
```

1.2 AKI

1.2.1 index.ts

```
testNapi.androidJNISayHello(1)

var djnniInterface = new testNapi2.TestDjinniWrapper()
var infoStore = new testNapi2.MyRecord(1234, "djinni test \n", "djinni", Gender.Man)
var a: string = djnniInterface.sayHello(infoStore)
this.message = a
```

1.2.2 index.d.ts

Additionally, in TS, we need to define the *.d.ts file as well.

```
export const androidJNISayHello: (a: number) => void
```

2. User code CPP

2.1 JNI

2.1.1 native-lib.cpp

This file is transformed to hello.cpp in 2.2.1 hello.cpp.

```cpp
#include <jni.h>
#include <string>
#include "android/log.h"

extern "C" JNIEXPORT jstring JNICALL
MainActivity.stringFromJNI(
        JNIEnv *env,
        jobject MainActivity /* this */) {
    std::string hello = "Hello from C++";
    return env->NewStringUTF( bytes: hello.c_str());
}

extern "C"
JNIEXPORT void JNICALL
Java_com_ruturaj_huawei_isayhello_MainActivity_androidJNISayHello(JNIEnv *env, jclass MainActivity clazz,
                                                                  jint i) {
    // TODO: implement androidJNISayHello()
    __android_log_print( prio: ANDROID_LOG_ERROR, tag: "RRR", fmt: "%d", i);
}
```

2.1.2 test_djinni_wrapper.cpp

This file is transformed to 2.2.2 test_djinni_wrapper.cpp.

```cpp
#include <cstring>
#include "header/ISayHello.h"
#include "header/MyRecord.h"

class TestDjinniWrapper : public test::hello::ISayHello {
private:
    std::string msg;
public:
    std::string sayHello(const test::hello::MyRecord &testInfo) {
        msg = "name: " + testInfo.name + "project: " + testInfo.project;
        //msg = "hahahah";
        return msg;
    }
};


std::shared_ptr<TestDjinniWrapper::ISayHello> TestDjinniWrapper::ISayHello::NewInstance() {
    return std::make_shared<TestDjinniWrapper>();
}
```

2.2 AKI

2.2.1 hello.cpp

This file is transformed from native-lib.cpp as in 2.1.1 native-lib.cpp.

```cpp
#include <string>
#include <aki/jsbind.h>
#include <hilog/log.h>

void androidJNISayHello(int i) {
    // TODO: implement androidJNISayHello()
    OH_LOG_Print(LOG_APP, LOG_ERROR, LOG_DOMAIN, "RRR", "%{public}d", i);
}

JSBIND_GLOBAL() {
    JSBIND_FUNCTION(androidJNISayHello);
}

JSBIND_ADDON(entry)
```

2.2.2 test_djinni_wrapper.cpp

This file is transformed from 2.1.2 test_djinni_wrapper.cpp.

```cpp
//
// Created by Ruturaj on 2023-09-01.
//
// Node APIs are not fully supported. To solve the compilation error of the interface cannot be found,
// please include "napi/native_api.h".

#include <cstring>
#include <aki/jsbind.h>
#include "header/ISayHello.h"
#include "header/MyRecord.h"

class TestDjinniWrapper : public test::hello::ISayHello
{
private:
    std::string msg;

public:
    TestDjinniWrapper() = default;

    std::string sayHello(const test::hello::MyRecord &testInfo) {
        msg = "name: " + testInfo.name + " project: " + testInfo.project;
        return msg;
    }
};

std::shared_ptr<TestDjinniWrapper::ISayHello> TestDjinniWrapper::ISayHello::NewInstance() {
    return std::make_shared<TestDjinniWrapper>();
}

JSBIND_CLASS(TestDjinniWrapper) {
    JSBIND_CONSTRUCTOR<>();
    JSBIND_METHOD(sayHello);
}

JSBIND_GLOBAL() { JSBIND_FUNCTION(TestDjinniWrapper::ISayHello::NewInstance); }

JSBIND_ADDON(entry2)
```

3. Djinni generated Java

3.1 JNI

3.1.1 ISayHello.java

Here, CppProxy nested class is completely ignored during the TS transformation as in 1.2.2 index.d.ts.

```java
package com.ruturaj.huawei.isayhello;// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni
import java.util.concurrent.atomic.AtomicBoolean;

1 inheritor
public abstract class ISayHello {
    no usages
    public static final int VERSION = 1;
    1 usage  1 implementation
    public abstract String sayHello(MyRecord testInfo);
    1 usage
    public static ISayHello newInstance() { return CppProxy.newInstance(); }
    1 usage
    private static final class CppProxy extends ISayHello {
        3 usages
        private final long nativeRef;
        2 usages
        private final AtomicBoolean destroyed = new AtomicBoolean( initialValue: false);
        no usages
        private CppProxy(long nativeRef) {
            if (nativeRef == 0) throw new RuntimeException("nativeRef is zero");
            this.nativeRef = nativeRef;
        }
        1 usage
        private native void nativeDestroy(long nativeRef);
        1 usage
        public void _djinni_private_destroy() {
            boolean destroyed = this.destroyed.getAndSet( newValue: true);
            if (!destroyed) nativeDestroy(this.nativeRef);
        }
```

```
        1 usage
        protected void finalize() throws Throwable {
            _djinni_private_destroy();
            super.finalize();
        }
        1 usage
        @Override
        public String sayHello(MyRecord testInfo) {
            assert !this.destroyed.get() : "trying to use a destroyed object";
            return native_sayHello(this.nativeRef, testInfo);
        }
        1 usage
        private native String native_sayHello(long _nativeRef, MyRecord testInfo);
        1 usage
        public static native ISayHello newInstance();
    }
}
```

3.1.2 MyRecord.java

```
package com.ruturaj.huawei.isayhello;// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni
import androidx.annotation.NonNull;
5 usages
public final class MyRecord {
    3 usages
    /*package*/ final int id;
    3 usages
    /*package*/ final String name;
    3 usages
    /*package*/ final String project;
    3 usages
    /*package*/ final Gender sex;
    1 usage
    public MyRecord(
            int id,
            String name,
            String project,
            Gender sex) {
        this.id = id;
        this.name = name;
        this.project = project;
        this.sex = sex;
    }
    no usages
    public int getId() { return id; }
    public String getName() { return name; }
    no usages
    public String getProject() { return project; }
    no usages
    public Gender getSex() { return sex; }
    @Override
    public String toString() {
        return "com.ruturaj.huawei.isayhello.MyRecord{" +
                "id=" + id +
                "," + "name=" + name +
                "," + "project=" + project +
                "," + "sex=" + sex +
                "}";
    }
}
```

3.1.3 Gender.java

```
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni
package com.ruturaj.huawei.isayhello;

4 usages
public enum Gender {
    1 usage
    Man,
    no usages
    Woman,
    ;
}
```

3.2 AKI

All the code in Java as in sections 3.1.1 ISayHello and 3.1.2 MyRecord and 3.1.3 Gender are transformed to definitions in 3.2.1 index.d.ts and Gender.java is also transformed in 3.2.2 index.ts.

3.2.1 index.d.ts

Additionally here, TestDjinniWrapper class which is a user code is added in the index.d.ts file so that the user defined code can be called in the index.ts as in the section 2.2.2 test_djinni_wrapper.cpp.

```
export enum Gender {
  Man = 0,
  Woman = 1
}

export class ISayHello {
  static readonly VERSION = 1

  sayHello: (testInfo: MyRecord) => string

  newInstance: () => ISayHello
}

export class MyRecord {
  id: number
  name: string
  project: string
  sex: Gender

  constructor(id: number, name: string, project: string, sex: Gender)

  getId: () => number

  getName: () => string

  getProject: () => string

  getSex: () => Gender

  toString: () => string
}

export class TestDjinniWrapper extends ISayHello {
  sayHello: (testInfo: MyRecord) => string
  newInstance: () => ISayHello
}
```

3.2.2 index.ts

```
enum Gender {
  Man,
  Woman
}
```

4. Djinni generated CPP

4.1 JNI

4.1.1 ISayHello.cpp

```cpp
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni

#include ...

namespace test {
    namespace hello {
        int32_t constexpr ISayHello::VERSION;
    }
}  // namespace test::hello
```

4.1.2 JNIISayHello.cpp

```
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni

#include "JNIISayHello.h"  // my header
#include "JNIMyRecord.h"
#include "support_lib/include/jni/Marshal.hpp"

namespace djinni_generated {

    JNIISayHello::JNIISayHello() : ::djinni::JniInterface<::test::hello::ISayHello, JNIISayHello>(
            cppProxyClassName: "com/ruturaj/huawei/isayhello/ISayHello$CppProxy") {}

    JNIISayHello::~JNIISayHello() = default;


    CJNIEXPORT void JNICALL
    Java_com_ruturaj_huawei_isayhello_ISayHello_00024CppProxy_nativeDestroy(JNIEnv *jniEnv,
                                                                            jobject ISayHello.CppProxy
                                                                            /*this*/,
                                                                            jlong nativeRef) {
        try {
            DJINNI_FUNCTION_PROLOGUE1(jniEnv, nativeRef);
            delete reinterpret_cast<::djinni::CppProxyHandle<::test::hello::ISayHello> *>(nativeRef);
        } JNI_TRANSLATE_EXCEPTIONS_RETURN(jniEnv,)
    }
/*
extern "C"*/ CJNIEXPORT jstring
    Java_com_ruturaj_huawei_isayhello_ISayHello_00024CppProxy_native_1sayHello(JNIEnv *jniEnv,
                                                                               jobject ISayHello.CppProxy
                                                                               /*this*/,
                                                                               jlong nativeRef,
                                                                               jobject MyRecord
                                                                               j_testInfo) {
```

```
        try {
            DJINNI_FUNCTION_PROLOGUE1(jniEnv, nativeRef);
            const auto &ref : const shared_ptr<...> & = ::djinni::objectFromHandleAddress<
                    ::test::hello::ISayHello>(
                    handle: nativeRef);
            auto r : string = ref->sayHello( testInfo: ::djinni_generated::JNIMyRecord::toCpp(jniEnv,
                                                                                              j: j_testInfo));
            return ::djinni::release( x: ::djinni::String::fromCpp(jniEnv, c: r));
        } JNI_TRANSLATE_EXCEPTIONS_RETURN(jniEnv, 0 /* value doesn't matter */)
    }

/*extern "C" */CJNIEXPORT jobject ISayHello
    Java_com_ruturaj_huawei_isayhello_ISayHello_00024CppProxy_newInstance(JNIEnv *jniEnv,
                                                                          jclass ISayHello.CppProxy
                                                                          clazz) {
        try {
            DJINNI_FUNCTION_PROLOGUE0(jniEnv);
            auto r : shared_ptr<ISayHello> = ::test::hello::ISayHello::NewInstance();
            return ::djinni::release( x: ::djinni_generated::JNIISayHello::fromCpp(jniEnv, c: r));
        } JNI_TRANSLATE_EXCEPTIONS_RETURN(jniEnv, 0 /* value doesn't matter */)
    }

}  // namespace djinni_generated
```

4.1.3 JNIMyRecord.cpp

```cpp
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni
#include "JNIMyRecord.h"  // my header
#include "JNIGender.h"
#include "support_lib/include/jni/Marshal.hpp"

namespace djinni_generated {
    JNIMyRecord::JNIMyRecord() = default;
    JNIMyRecord::~JNIMyRecord() = default;
    auto JNIMyRecord::fromCpp(JNIEnv *jniEnv, const CppType &c) -> ::djinni::LocalRef<JniType> {
        const auto &data : JNIMyRecord const & = ::djinni::JniClass<JNIMyRecord>::get();
        auto r = ::djinni::LocalRef<JniType>{ localRef: jniEnv->NewObject( clazz: data.clazz.get(),
                                              methodID: data.jconstructor,
                                                        ::djinni::get( x: ::djinni::I32::fromCpp(
                                                                jniEnv,  c: c.id)),
                                                        ::djinni::get(
                                                                x: ::djinni::String::fromCpp(
                                                                        jniEnv,  c: c.name)),
                                                        ::djinni::get(
                                                                x: ::djinni::String::fromCpp(
                                                                        jniEnv,  c: c.project)),
                                                        ::djinni::get(
                                                                x: ::djinni_generated::JNIGender
                                                                ::fromCpp(
                                                                        jniEnv,  c: c.sex)))};
        ::djinni::jniExceptionCheck( env: jniEnv);
        return r;
    }
    auto JNIMyRecord::toCpp(JNIEnv *jniEnv, JniType j) -> CppType {
        ::djinni::JniLocalScope jscope( p_env: jniEnv,  capacity: 5);
        assert(j != nullptr);
        const auto &data : JNIMyRecord const & = ::djinni::JniClass<JNIMyRecord>::get();
        return { id_: ::djinni::I32::toCpp(jniEnv,  j: jniEnv->GetIntField( obj: j,  fieldID: data.field_id)),
                 name_: ::djinni::String::toCpp(jniEnv,
                                                j: (jstring) jniEnv->GetObjectField( obj: j,
                                                                                     fieldID: data.field_name)),
                 project_: ::djinni::String::toCpp(jniEnv,
                                                   j: (jstring) jniEnv->GetObjectField( obj: j,
                                                                                        fieldID: data.field_project)),
                 sex_: ::djinni_generated::JNIGender::toCpp(jniEnv,
                                                            j: jniEnv->GetObjectField( obj: j,
                                                                                       fieldID: data.field_sex))};
    }

}  // namespace djinni_generated
```

4.2 AKI

All these files as in sections 4.1.1 ISayHello.cpp and 4.1.2 JNIISayHello.cpp and 4.1.3 JNIMyRecord.cpp have been ignored, as they are dependent on Djinni support libraries and those support libraries contain JVM (Java Virtual Machine) usage, hence no direct transformation is available.

5. Djinni generated HPP for CPP

5.1 JNI

5.1.1 JNIISayHello.h

```cpp
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni

#pragma once
#include "support_lib/include/jni/djinni_support.hpp"
#include "header/ISayHello.h"

namespace djinni_generated {
    class JNIISayHello final
            : ::djinni::JniInterface<::test::hello::ISayHello, JNIISayHello> {
    public:
        using CppType = std::shared_ptr<::test::hello::ISayHello>;
        using CppOptType = std::shared_ptr<::test::hello::ISayHello>;
        using JniType = jobject;

        using Boxed = JNIISayHello;

        ~JNIISayHello();

        static CppType toCpp(JNIEnv *jniEnv, JniType j) {
            return ::djinni::JniClass<JNIISayHello>::get()._fromJava(jniEnv, j);
        }

        static ::djinni::LocalRef<JniType> fromCppOpt(JNIEnv *jniEnv, const CppOptType &c) {
            return {jniEnv, localRef: ::djinni::JniClass<JNIISayHello>::get()._toJava(jniEnv, c)};
        }

        static ::djinni::LocalRef<JniType>
        fromCpp(JNIEnv *jniEnv, const CppType &c) { return fromCppOpt(jniEnv, c); }

    private:
        JNIISayHello();

        friend ::djinni::JniClass<JNIISayHello>;
        friend ::djinni::JniInterface<::test::hello::ISayHello, JNIISayHello>;

    };

}  // namespace djinni_generated
```

5.1.2 JNIMyRecord.h

```cpp
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni
#pragma once
#include "support_lib/include/jni/djinni_support.hpp"
#include "header/MyRecord.h"

namespace djinni_generated {
    class JNIMyRecord final {
    public:
        using CppType = ::test::hello::MyRecord;
        using JniType = jobject;

        using Boxed = JNIMyRecord;

        ~JNIMyRecord();

        static CppType toCpp(JNIEnv *jniEnv, JniType j);

        static ::djinni::LocalRef<JniType> fromCpp(JNIEnv *jniEnv, const CppType &c);
    private:
        JNIMyRecord();

        friend ::djinni::JniClass<JNIMyRecord>;

        const ::djinni::GlobalRef<jclass> clazz{
                ::djinni::jniFindClass( name: "com/ruturaj/huawei/isayhello/MyRecord")};
        const jmethodID jconstructor{::djinni::jniGetMethodID( clazz: clazz.get(), name: "<init>",
                sig: "(ILjava/lang/String;Ljava/lang/String;Lcom/ruturaj/huawei/isayhello/Gender;)V")};
        const jfieldID field_id{::djinni::jniGetFieldID( clazz: clazz.get(), name: "id", sig: "I")};
        const jfieldID field_name{
                ::djinni::jniGetFieldID( clazz: clazz.get(), name: "name", sig: "Ljava/lang/String;")};
        const jfieldID field_project{
                ::djinni::jniGetFieldID( clazz: clazz.get(), name: "project", sig: "Ljava/lang/String;")};
        const jfieldID field_sex{
                ::djinni::jniGetFieldID( clazz: clazz.get(), name: "sex",
                                         sig: "Lcom/ruturaj/huawei/isayhello/Gender;")};
    };

}  // namespace djinni_generated
```

5.1.3 JNIGender.h

```
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni

#pragma once

#include ...

namespace djinni_generated {

    class JNIGender final : ::djinni::JniEnum {
    public:
        using CppType = ::test::hello::Gender;
        using JniType = jobject;

        using Boxed = JNIGender;

        static CppType toCpp(JNIEnv *jniEnv, JniType j) {
            return static_cast<CppType>(::djinni::JniClass<JNIGender>::get().ordinal( env: jniEnv,  obj: j));
        }

        static ::djinni::LocalRef<JniType> fromCpp(JNIEnv *jniEnv, CppType c) {
            return ::djinni::JniClass<JNIGender>::get().create( env: jniEnv,  value: static_cast<jint>(c));
        }

    private:
        JNIGender() : JniEnum( name: "com/ruturaj/huawei/isayhello/Gender") {}

        friend ::djinni::JniClass<JNIGender>;
    };

}  // namespace djinni_generated
```

5.2 AKI

All these files as in sections 5.1.1 JNIISayHello.h and 5.1.2 JNIMyRecord.h and 5.1.3 JNIGender.h have been ignored, as they are dependent on Djinni support libraries and those support libraries contain JVM (Java Virtual Machine) usage, hence no direct transformation is available.

6. Djinni generated HPP for user code

6.1 JNI

6.1.1 ISayHello.h

This file is transformed to section 6.2.1 ISayHello.h.

```
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni

#pragma once

#include <cstdint>
#include <memory>
#include <string>
#include <stdint.h>

namespace test {
    namespace hello {

        struct MyRecord;

        class ISayHello {
        public:
            virtual ~ISayHello() {}

            static constexpr int32_t VERSION = 1;

            virtual std::string sayHello(const MyRecord &testInfo) = 0;

            static std::shared_ptr<ISayHello> NewInstance();
        };

    }
}  // namespace test::hello
```

6.1.2 MyRecord.h

This file is transformed to section 6.2.2 MyRecord.h.

```
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni

#pragma once

#include "Gender.h"
#include <cstdint>
#include <string>
#include <utility>

namespace test {
    namespace hello {

        struct MyRecord final {
            int32_t id;
            std::string name;
            std::string project;
            Gender sex;

            MyRecord(int32_t id_,
                     std::string name_,
                     std::string project_,
                     Gender sex_)
                    : id(std::move(id_)), name(std::move(name_)), project(std::move(project_)),
                      sex(std::move(sex_)) {}
        };

    }
}  // namespace test::hello
```

6.1.3 Gender.h

This file is transformed to section 6.2.3 Gender.h.

```cpp
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni

#pragma once

#include <functional>

namespace test {
    namespace hello {

        enum class Gender : int {
            Man,
            Woman,
        };

    }
}  // namespace test::hello

namespace std {

    template<>
    struct hash<::test::hello::Gender> {
        size_t operator()(::test::hello::Gender type) const {
            return std::hash<int>()( v: static_cast<int>(type));
        }
    };

}  // namespace std
```

6.2 AKI

6.2.1 ISayHello.h

This file is transformed from section 6.1.1 ISayHello.h.

```cpp
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni

#pragma once

#include <cstdint>
#include <memory>
#include <string>
#include <stdint.h>
#include "aki/jsbind.h"

namespace test {
namespace hello {

class MyRecord;

class ISayHello
{
public:
    virtual ~ISayHello() {}

    static constexpr int32_t VERSION = 1;

    std::string sayHello(const MyRecord &testInfo);

    static std::shared_ptr<ISayHello> NewInstance();
};

} // namespace hello
} // namespace test
```

6.2.2 MyRecord.h

This file is transformed from section 6.1.2 MyRecord.h.

```cpp
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni

#pragma once

#include "Gender.h"
#include <cstdint>
#include <string>
#include <utility>
#include "aki/jsbind.h"

namespace test {
namespace hello {

class MyRecord
{
public:
    MyRecord() = default;

    int32_t id;
    std::string name;
    std::string project;
    Gender sex;

    MyRecord(int32_t id_, std::string name_, std::string project_, Gender sex_)
        : id(id_), name(std::move(name_)), project(std::move(project_)), sex(sex_) {}
};

JSBIND_CLASS(MyRecord) {
    JSBIND_CONSTRUCTOR<int32_t, std::string, std::string, Gender>();
    JSBIND_PROPERTY(id);
    JSBIND_PROPERTY(name);
    JSBIND_PROPERTY(project);
    JSBIND_PROPERTY(sex);
}

} // namespace hello
} // namespace test
```

6.2.3 Gender.h

This file is transformed from section 6.1.3 Gender.h.

```cpp
// AUTOGENERATED FILE - DO NOT MODIFY!
// This file generated by Djinni from info.djinni

#pragma once

#include <functional>
#include "aki/jsbind.h"

namespace test {
namespace hello {

enum Gender {
    Man,
    Woman,
};

JSBIND_ENUM(Gender) {
    JSBIND_ENUM_VALUE(Man);
    JSBIND_ENUM_VALUE(Woman);
}
} // namespace hello
} // namespace test
```

7. CMakeLists.txt file

7.1 JNI

```
# For more information about using CMake with Android Studio, read the
# documentation: https://d.android.com/studio/projects/add-native-code.html.
# For more examples on how to use CMake, see https://github.com/android/ndk-samples.

# Sets the minimum CMake version required for this project.
cmake_minimum_required(VERSION 3.22.1)

# …
project("isayhello")

# …
add_library(${CMAKE_PROJECT_NAME} SHARED
        # List C/C++ source files with relative paths to this CMakeLists.txt.
        ISayHello.cpp
        JNIISayHello.cpp
        JNIMyRecord.cpp
        support_lib/djinni_support.cpp
        support_lib/djinni_main.cpp
        test_djinni_wrapper.cpp
        native-lib.cpp)

# …
target_link_libraries(${CMAKE_PROJECT_NAME}
        # List libraries link to the target library
        android
        log)
```

7.2 AKI

```
# the minimum version of CMake.
cmake_minimum_required(VERSION 3.4.1)
project(ISayHello)

set(NATIVERENDER_ROOT_PATH ${CMAKE_CURRENT_SOURCE_DIR})

include_directories(${NATIVERENDER_ROOT_PATH}
                    ${NATIVERENDER_ROOT_PATH}/include)

add_library(entry SHARED hello.cpp)
add_library(entry2 SHARED test_djinni_wrapper.cpp)
add_subdirectory(aki)
target_link_libraries(entry PUBLIC aki_jsbind)
target_link_libraries(entry2 PUBLIC aki_jsbind)
```

8. Notes

For mapping instructions, please refer ANDROIDDJINNIJNI 2 AKI.XLSX file attached in the same folder.